Query Processing (Chapter 23 – Connelly)

CSE 341
Introduction to Database Systems
Winter 2024
© Royal Military College of Canada

Introduction

- When the relational model was first launched commercially, one of the major criticisms often cited was inadequate performance of queries.
- Since then, a significant amount of research has been devoted to developing highly efficient algorithms for processing queries.
- There are many ways in which a complex query can be performed, and one of the aims of query processing is to determine which one is the most cost-effective.

Pearson Education © 2014

Introduction

- With declarative languages such as SQL, the user specifies *what* data is required rather than *how* it is to be retrieved.
- Relieves user of knowing what constitutes good execution strategy.

Introduction

- Also gives DBMS more control over system performance.

- Two main techniques for query optimization:
  - heuristic rules that order operations in a query;
  - comparing different strategies based on relative costs, and selecting one that minimizes resource usage.

- Disk access tends to be dominant cost in query processing for centralized DBMS.

Pearson Education © 2014

Query Processing (QP)

Activities involved in retrieving data from the database.

- Aims of QP:
  - transform query written in high-level language (e.g. SQL), into correct and efficient execution strategy expressed in low-level language (Relational Algebra - RA);
  - execute strategy to retrieve required data.

Pearson Education © 2014

Query Optimization (QO)

Activity of choosing an efficient execution strategy for processing query.

- As there are many equivalent transformations of same high-level query, aim of QO is to choose one that minimizes resource usage.
- Generally, reduce total execution time of query.
- May also reduce response time of query.
- Problem computationally intractable with large number of relations, so strategy adopted is reduced to finding near optimum solution.

Pearson Education © 2014

Example 23.1 - Different Strategies

Find all Managers who work at a London branch.

```
SELECT *
FROM Staff s, Branch b
WHERE s.branchNo = b.branchNo AND
(s.position = 'Manager' AND b.city = 'London');
```

Pearson Education © 2014

Example 23.1 - Different Strategies

- The three equivalent RA queries are:

(1) $\sigma_{(\text{position}=\text{'Manager'}) \wedge (\text{city}=\text{'London'}) \wedge (\text{Staff.branchNo}=\text{Branch.branchNo})}(\text{Staff} \times \text{Branch})$

(2) $\sigma_{(\text{position}=\text{'Manager'}) \wedge (\text{city}=\text{'London'})}(\text{Staff} \bowtie_{\text{Staff.branchNo}=\text{Branch.branchNo}} \text{Branch})$

(3) $(\sigma_{\text{position}=\text{'Manager'}}(\text{Staff})) \bowtie_{\text{Staff.branchNo}=\text{Branch.branchNo'}} (\sigma_{\text{city}=\text{'London'}}(\text{Branch}))$

Pearson Education © 2014

Example 23.1 - Different Strategies

- Assume:
  - 1000 tuples in Staff; 50 tuples in Branch;
  - 50 Managers; 5 London branches;
  - no indexes or sort keys;
  - results of any intermediate operations stored on disk;
  - cost of the final write is ignored;
  - tuples are accessed one at a time.

Pearson Education © 2014

Example 23.1 - Cost Comparison

- Cost (in disk accesses) are:

(1) (1000 + 50) + 2*(1000 * 50) = 101 050

(2) 2*1000 + (1000 + 50) = 3 050

(3) 1000 + 2*50 + 5 + (50 + 5) = 1 160

- Cartesian product and join operations much more expensive than selection, and third option significantly reduces size of relations being joined together.

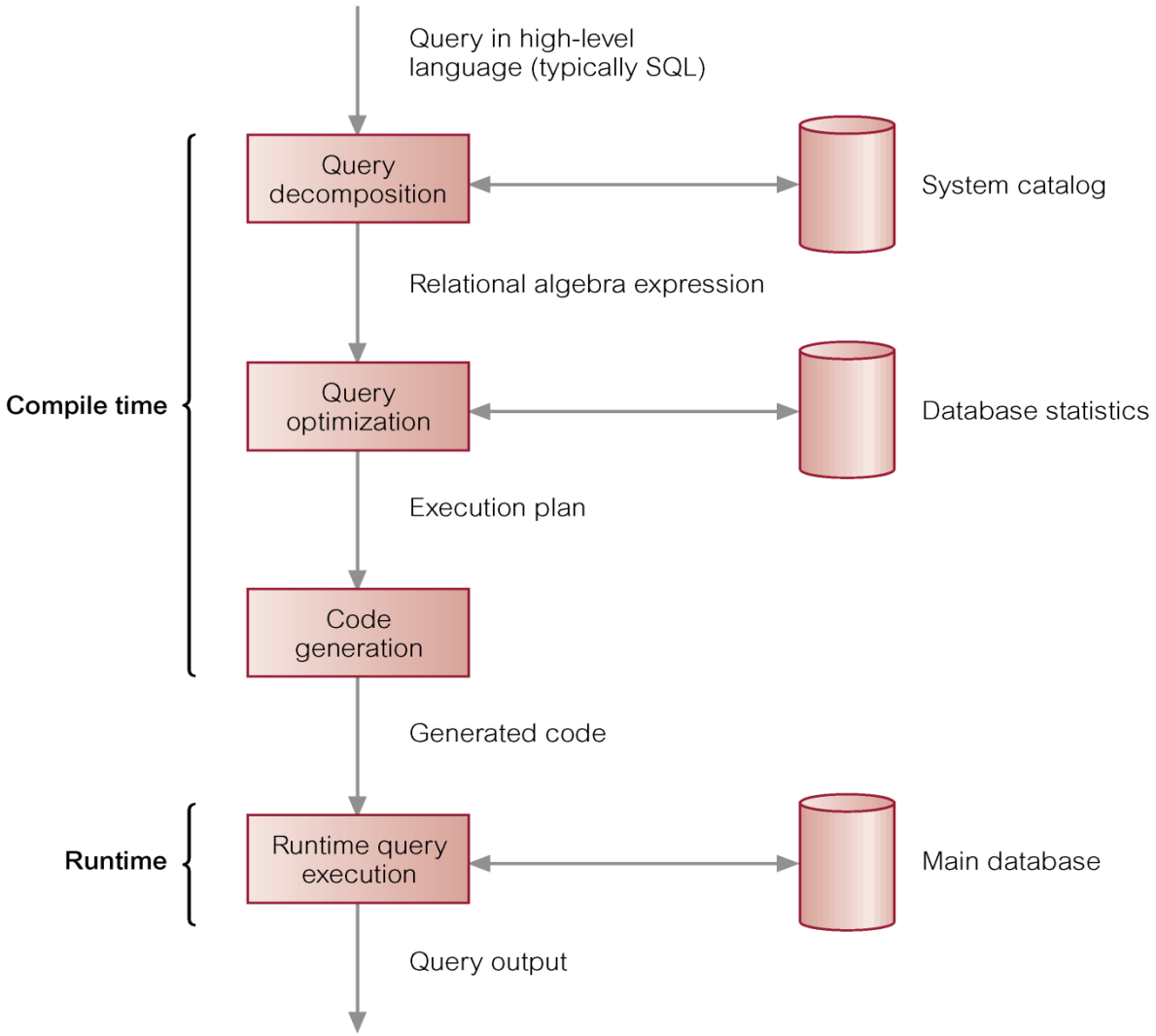
Pearson Education © 2014

Phases of Query Processing

- QP has four main phases:
  - decomposition (consisting of parsing and validation);
  - optimization;
  - code generation;
  - execution.

Pearson Education © 2014

Phases of Query Processing



Pearson Education © 2014

Dynamic versus Static Optimization

- Two times when first three phases of QP can be carried out:
  - dynamically every time query is run;
  - statically when query is first submitted.
- Advantages of dynamic QO arise from fact that information is up to date.
- Disadvantages are that performance of query is affected, time may limit finding optimum strategy.

Pearson Education © 2014

Dynamic versus Static Optimization

- Advantages of static QO are removal of runtime overhead, and more time to find optimum strategy.
- Disadvantages arise from fact that chosen execution strategy may no longer be optimal when query is run.
- Could use a hybrid approach to overcome this.

Pearson Education © 2014

Query Decomposition

Query Decomposition

- Aims are to transform high-level query into RA query and check that query is syntactically and semantically correct.
- Typical stages are:
  - analysis,
  - normalization,
  - semantic analysis,
  - simplification,
  - query restructuring.

Pearson Education © 2014

Analysis

- Analyze query lexically and syntactically using compiler techniques.
- Verify relations and attributes exist.
- Verify operations are appropriate for object type.

Pearson Education © 2014

Analysis - Example

```
SELECT staffNumber
FROM Staff
WHERE position > 10;
```

- This query would be rejected on two grounds:
  - staffNumber is not defined for Staff relation (should be staffNo).
  - Comparison '>10' is incompatible with type position, which is variable character string.

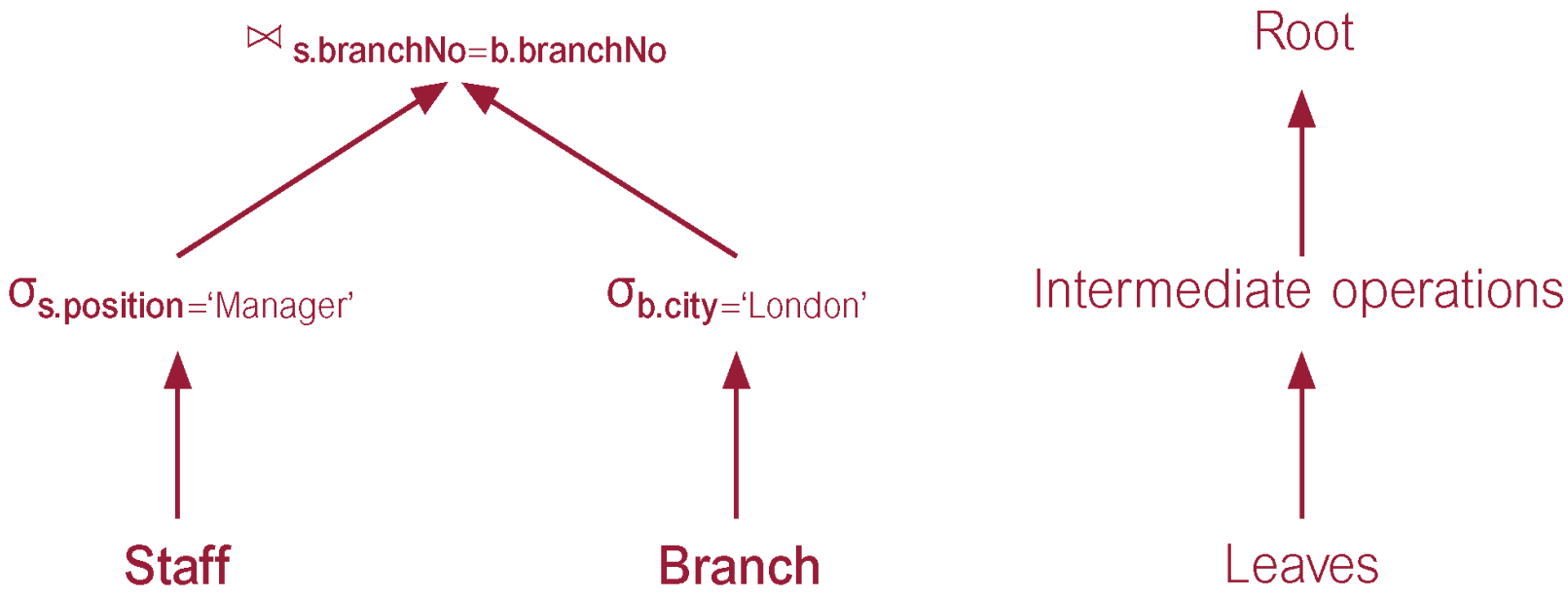
Pearson Education © 2014

Analysis

- Finally, query transformed into some internal representation more suitable for processing.
- Some kind of query tree is typically chosen, constructed as follows:
  - Leaf node created for each base relation.
  - Non-leaf node created for each intermediate relation produced by RA operation.
  - Root of tree represents query result.
  - Sequence is directed from leaves to root.

Pearson Education © 2014

Example 23.1 - R.A.T. (Relational Algebra Tree)



Pearson Education © 2014

Normalization

- Converts query into a normalized form for easier manipulation.
- Predicate (in SQL, the WHERE condition) can be converted into one of two forms:

<u>Conjunctive normal form:</u>

(position = 'Manager' $\vee$ salary > 20000) $\wedge$ (branchNo = 'B003')

<u>Disjunctive normal form:</u>

(position = 'Manager' $\wedge$ branchNo = 'B003') $\vee$

(salary > 20000 $\wedge$ branchNo = 'B003')

Pearson Education © 2014

Semantic Analysis

- Rejects normalized queries that are incorrectly formulated or contradictory.
- Query is <u>incorrectly formulated</u> if components do not contribute to generation of result.
- Query is <u>contradictory</u> if its predicate cannot be satisfied by any tuple.
- Algorithms to determine correctness exist only for queries that do not contain disjunction and negation.

Pearson Education © 2014

Semantic Analysis

- For these queries, could construct:
  - A relation connection graph.
  - Normalized attribute connection graph.

Pearson Education © 2014

Semantic Analysis: Relation connection graph

- Create node for each relation and node for result. Then, create edges between two nodes that represent a join, and edges between nodes that represent the source of Projection operations.
- If not fully connected, query is incorrectly formulated.

Pearson Education © 2014

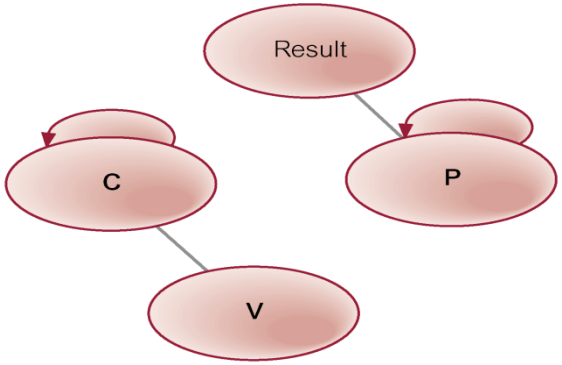
Example 23.2 - Checking Semantic Correctness

```
SELECT p.propertyNo, p.street
FROM Client c, Viewing v, PropertyForRent p
WHERE   c.clientNo = v.clientNo AND
        c.maxRent >= 500 AND
        c.prefType = 'Flat' AND p.ownerNo = 'CO93';
```

- Relation connection graph not fully connected, so query is not correctly formulated.
- Have omitted the join condition (v.propertyNo = p.propertyNo) .

Pearson Education © 2014

Example 23.2 - Checking Semantic Correctness



(a)

Relation Connection graph

Semantic Analysis - Normalized Attribute Connection Graph

- Create node for each reference to an attribute, or constant 0.
- Create directed edge between nodes that represent a join, and directed edge between attribute node and 0 node that represents selection.
- Weight edges $a \rightarrow b$ with value c, if it represents inequality condition ($a \leq b + c$); weight edges $0 \rightarrow a$ with -c, if it represents inequality condition ($a \geq c$).
- If graph has cycle for which valuation sum is negative, query is contradictory.

Pearson Education © 2014

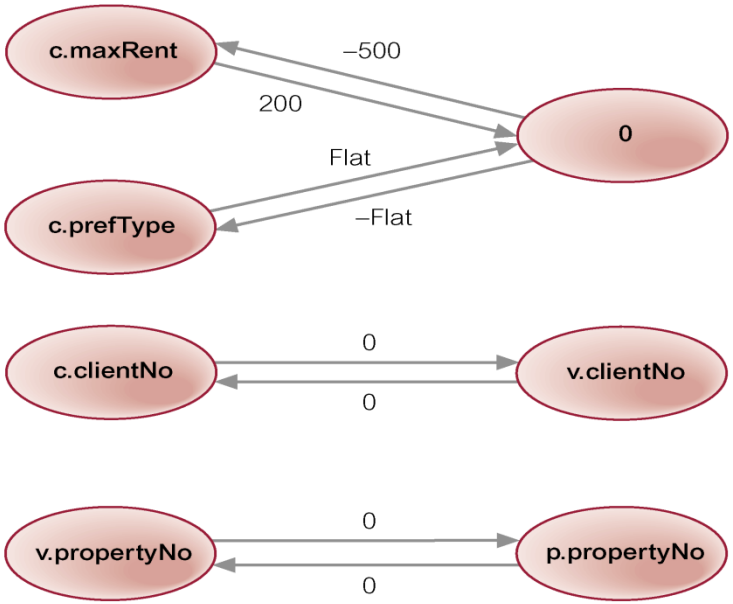
Example 23.2 - Checking Semantic Correctness

```
SELECT p.propertyNo, p.street
FROM Client c, Viewing v, PropertyForRent p
WHERE   c.maxRent > 500 AND
        c.clientNo = v.clientNo AND
        v.propertyNo = p.propertyNo AND
        c.prefType = 'Flat' AND c.maxRent < 200;
```

- Normalized attribute connection graph has cycle between nodes c.maxRent and 0 with negative valuation sum, so query is contradictory.

Pearson Education © 2014

Example 23.2 - Checking Semantic Correctness



(b)

Normalized attribute connection graph

Simplification

- Detects redundant qualifications,
- eliminates common sub-expressions,
- transforms query to semantically equivalent but more easily and efficiently computed form.

Pearson Education © 2014

Transformation Rules for RA Operations

Conjunctive Selection operations can cascade into individual Selection operations (and vice versa).

$$\sigma_{p \wedge q \wedge r}(R) = \sigma_p(\sigma_q(\sigma_r(R)))$$

- **Sometimes referred to as cascade of Selection.**

$$\sigma_{branchNo='B003' \wedge salary>15000}(Staff) = \sigma_{branchNo='B003'}(\sigma_{salary>15000}(Staff))$$

Pearson Education © 2014

Heuristical Approach to Query Optimization

Uses transformation rules to convert one relational algebra expression into an equivalent form that is known to be more efficient.

Transformation Rules for RA Operations

Commutativity of Selection.

$$\sigma_p(\sigma_q(R)) = \sigma_q(\sigma_p(R))$$

- For example:

$$\sigma_{branchNo='B003'}(\sigma_{salary>15000}(Staff)) = \sigma_{salary>15000}(\sigma_{branchNo='B003'}(Staff))$$

Pearson Education © 2014

Transformation Rules for RA Operations

In a sequence of Projection operations, only the last in the sequence is required.

$$\Pi_L \Pi_M \ldots \Pi_N(R) = \Pi_L (R)$$

- For example:

$$\Pi_{lName} \Pi_{branchNo,\ lName}(Staff) = \Pi_{lName} (Staff)$$

Pearson Education © 2014

Transformation Rules for RA Operations

Commutativity of Selection and Projection.

- If predicate p involves only attributes in projection list, Selection and Projection operations commute:

$$\Pi_{Ai, \ldots, Am}(\sigma_p(R)) = \sigma_p(\Pi_{Ai, \ldots, Am}(R))$$
$$\text{where } p \in \{A_1, A_2, \ldots, A_m\}$$

- For example:

$$\Pi_{fName, lName}(\sigma_{lName='Beech'}(Staff)) = \sigma_{lName='Beech'}(\Pi_{fName,lName}(Staff))$$

Pearson Education © 2014

Transformation Rules for RA Operations

Commutativity of Theta join (and Cartesian product).

$R \bowtie_p S = S \bowtie_p R$

$R \times S = S \times R$

Rule also applies to Equijoin and Natural join. For example:

$Staff \bowtie_{staff.branchNo=branch.branchNo} Branch =$

$Branch \bowtie_{staff.branchNo=branch.branchNo} Staff$

Pearson Education © 2014

Transformation Rules for RA Operations

Commutativity of Selection and Theta join (or Cartesian product).

- If selection predicate involves only attributes of one of the join relations, Selection and Join (or Cartesian product) operations commute:

$$\sigma_p(R \bowtie_r S) = (\sigma_p(R)) \bowtie_r S$$

$$\sigma_p(R \times S) = (\sigma_p(R)) \times S$$

where $p \in \{A_1, A_2, \ldots, A_n\}$

Pearson Education © 2014

Transformation Rules for RA Operations

- If selection predicate is conjunctive predicate having form ($p \wedge q$), where p only involves attributes of R, and q only attributes of S, Selection and Theta join operations commute as:

$$\sigma_{p \wedge q}(R \bowtie_r S) = (\sigma_p(R)) \bowtie_r (\sigma_q(S))$$

$$\sigma_{p \wedge q}(R \times S) = (\sigma_p(R)) \times (\sigma_q(S))$$

Pearson Education © 2014

Transformation Rules for RA Operations

- For example:

$$\sigma_{\text{position='Manager'} \wedge \text{city='London'}}(\text{Staff} \bowtie_{\text{Staff.branchNo=Branch.branchNo}} \text{Branch}) =$$

$$(\sigma_{\text{position='Manager'}}(\text{Staff})) \bowtie_{\text{Staff.branchNo=Branch.branchNo}} (\sigma_{\text{city='London'}}(\text{Branch}))$$

Pearson Education © 2014

Transformation Rules for RA Operations

Commutativity of Projection and Theta join (or Cartesian product).

- If projection list is of form $L = L_1 \cup L_2$, where L_1 only has attributes of R, and L_2 only has attributes of S, provided join condition only contains attributes of L, Projection and Theta join commute:

$$\Pi_{L1 \cup L2}(R \bowtie_r S) = (\Pi_{L1}(R)) \bowtie_r (\Pi_{L2}(S))$$

Pearson Education © 2014

Transformation Rules for RA Operations

- If join condition contains additional attributes not in L ($M = M_1 \cup M_2$ where M_1 only has attributes of R, and M_2 only has attributes of S), a final projection operation is required:

$$\Pi_{L1 \cup L2}(R \bowtie_r S) = \Pi_{L1 \cup L2}((\Pi_{L1 \cup M1}(R)) \bowtie_r (\Pi_{L2 \cup M2}(S)))$$

$$\Pi_{\text{position, city}}(\text{Staff} \bowtie_{\text{Staff.branchNo}=\text{Branch.branchNo}} \text{Branch}) =$$

$$\Pi_{\text{position, city}}((\Pi_{\text{position, branchNo}}(\text{Staff})) \bowtie_{\text{Staff.branchNo}=\text{Branch.branchNo}} (\Pi_{\text{city, branchNo}}(\text{Branch})))$$

Pearson Education © 2014

Transformation Rules for RA Operations

- For example:

$$\Pi_{\text{position,city,branchNo}}(\text{Staff} \bowtie_{\text{Staff.branchNo=Branch.branchNo}} \text{Branch}) =$$
$$(\Pi_{\text{position, branchNo}}(\text{Staff})) \bowtie_{\text{Staff.branchNo=Branch.branchNo}} ($$
$$\Pi_{\text{city, branchNo}} (\text{Branch}))$$

- and using the latter rule:

$$\Pi_{\text{position, city}}(\text{Staff} \bowtie_{\text{Staff.branchNo=Branch.branchNo}} \text{Branch}) =$$
$$\Pi_{\text{position, city}} ((\Pi_{\text{position, branchNo}}(\text{Staff})) \bowtie_{\text{Staff.branchNo=Branch.branchNo}}$$
$$(\Pi_{\text{city, branchNo}} (\text{Branch})))$$

Pearson Education © 2014

Transformation Rules for RA Operations

Commutativity of Union and Intersection (but not set difference).

$R \cup S = S \cup R$

$R \cap S = S \cap R$

Pearson Education © 2014

Transformation Rules for RA Operations

Commutativity of Selection and set operations (Union, Intersection, and Set difference).

$\sigma_p(R \cup S) = \sigma_p(S) \cup \sigma_p(R)$

$\sigma_p(R \cap S) = \sigma_p(S) \cap \sigma_p(R)$

$\sigma_p(R - S) = \sigma_p(S) - \sigma_p(R)$

Pearson Education © 2014

Transformation Rules for RA Operations

Commutativity of Projection and Union.

$$\Pi_L(R \cup S) = \Pi_L(S) \cup \Pi_L(R)$$

Associativity of Union and Intersection (but not Set difference).

$$(R \cup S) \cup T = S \cup (R \cup T)$$

$$(R \cap S) \cap T = S \cap (R \cap T)$$

Pearson Education © 2014

Transformation Rules for RA Operations

Associativity of Theta join (and Cartesian product).

- Cartesian product and Natural join are always associative:

  $(R \bowtie S) \bowtie T = R \bowtie (S \bowtie T)$

  $(R \times S) \times T = R \times (S \times T)$

- If join condition q involves attributes only from S and T, then Theta join is associative:

  $(R \bowtie_p S) \bowtie_{q \wedge r} T = R \bowtie_{p \wedge r} (S \bowtie_q T)$

Pearson Education © 2014

Transformation Rules for RA Operations

- For example:

$$(\text{Staff} \bowtie_{\text{Staff.staffNo=PropertyForRent.staffNo}} \text{PropertyForRent}) \bowtie_{\text{ownerNo=Owner.ownerNo} \wedge \text{staff.lName=Owner.lName}} \text{Owner} =$$

$$\text{Staff} \bowtie_{\text{staff.staffNo=PropertyForRent.staffNo} \wedge \text{staff.lName=lName}} (\text{PropertyForRent} \bowtie_{\text{ownerNo}} \text{Owner})$$

Pearson Education © 2014

Example 23.3 Use of Transformation Rules

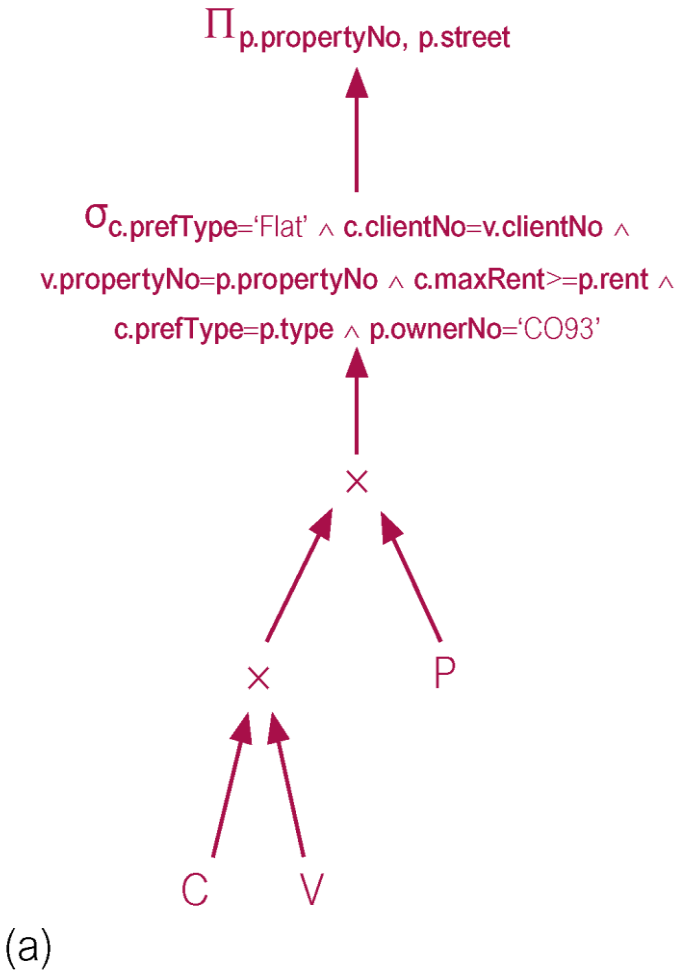
For prospective renters of flats, find properties that match requirements and owned by CO93.

```
SELECT p.propertyNo, p.street
FROM Client c, Viewing v, PropertyForRent p
WHERE   c.prefType = 'Flat' AND
        c.clientNo = v.clientNo AND
        v.propertyNo = p.propertyNo AND
        c.maxRent >= p.rent AND
        c.prefType = p.type AND
        p.ownerNo = 'CO93';
```

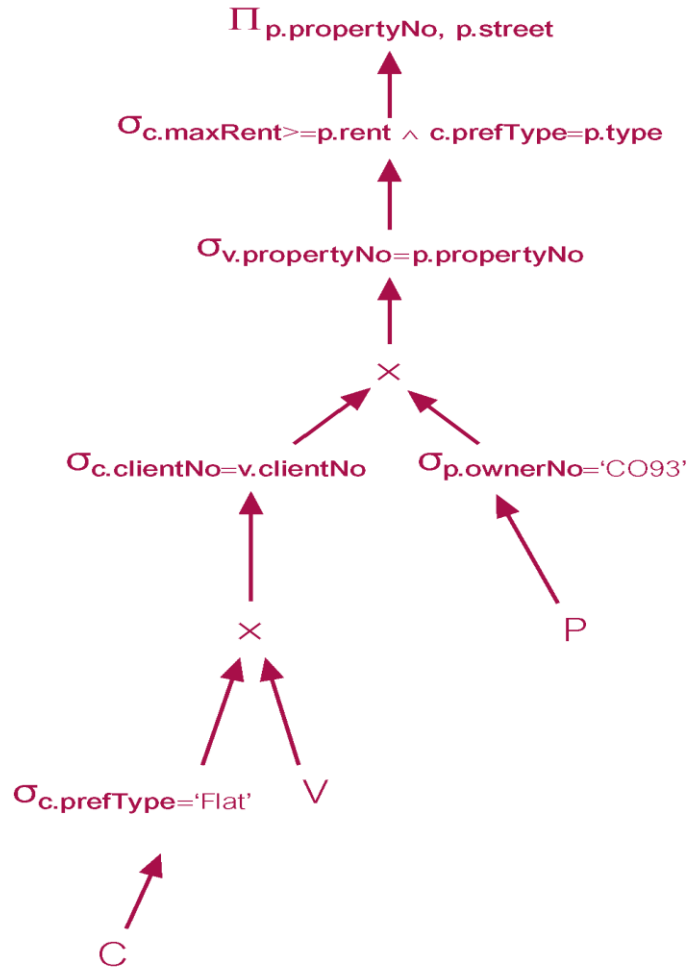
$$\Pi_{\text{p.propertyNo, p.street}}(\sigma_{\text{c.prefType= 'Flat'} \wedge \text{c.clientNo=v.clientNo} \wedge \text{v.propertyNo=p.propertyNo} \wedge \text{c.maxRent>=p.rent} \wedge \text{c.prefType=p.type} \wedge \text{p.ownerNo='CO93'}}((c \times v) \times p))$$

Pearson Education © 2014

Example 23.3 Use of Transformation Rules



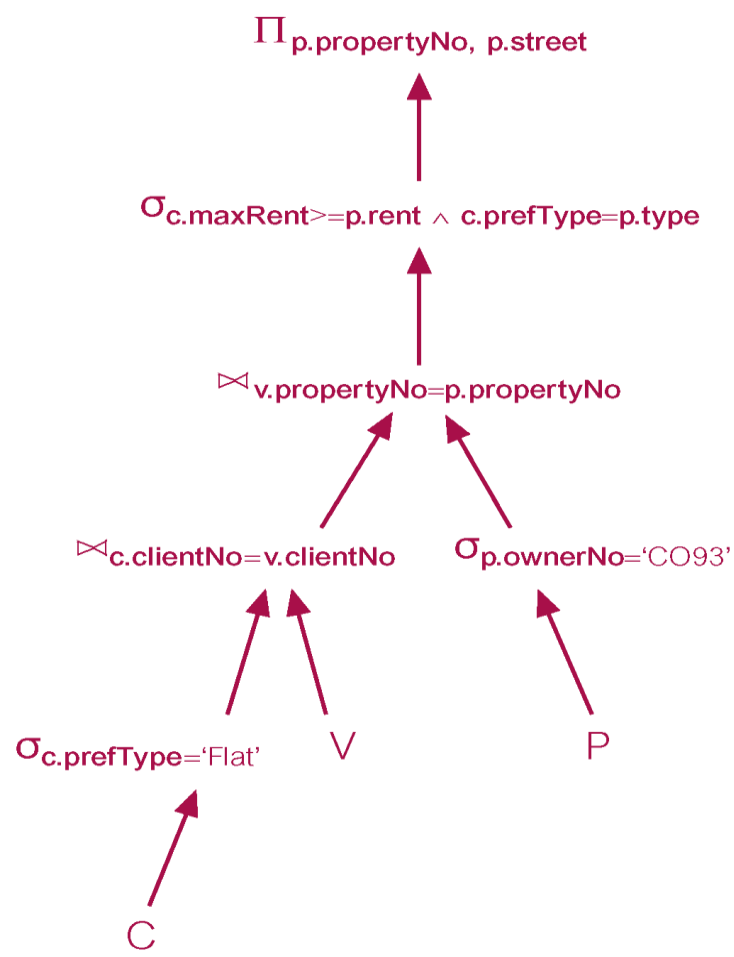
(a)



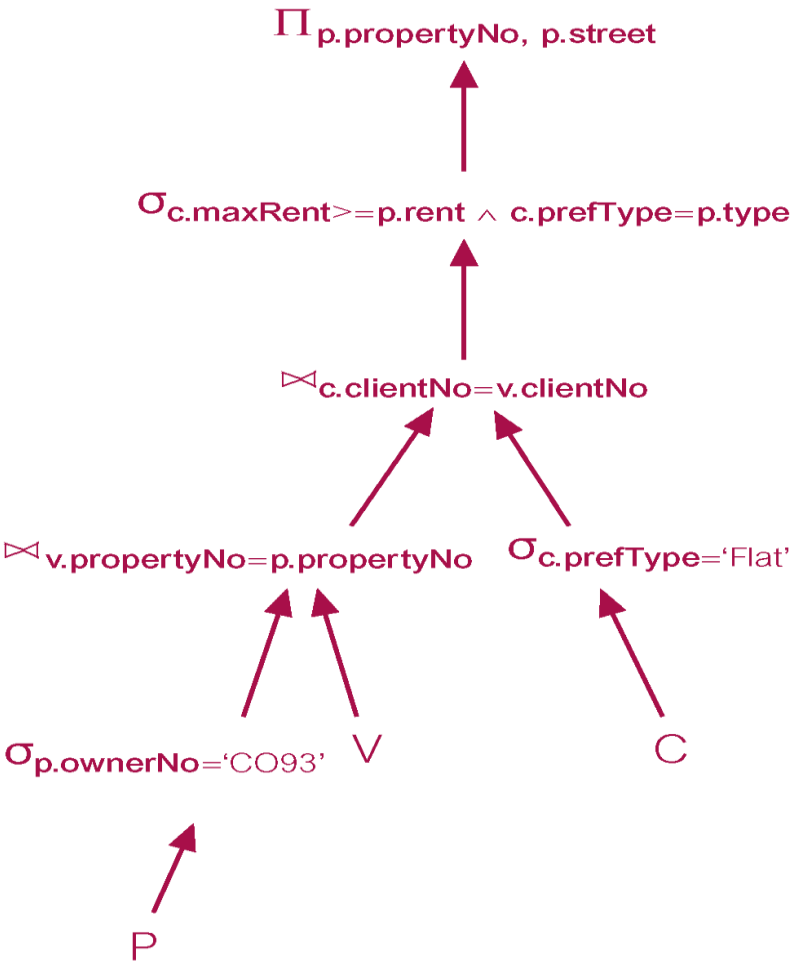
(b)

$$\Pi_{\text{p.propertyNo, p.street}}(\sigma_{\text{c.prefType= 'Flat'} \wedge \text{c.clientNo=v.clientNo} \wedge \text{v.propertyNo=p.propertyNo} \wedge \text{c.maxRent>=p.rent} \wedge \text{c.prefType=p.type} \wedge \text{p.ownerNo='C093'}}((c \times v) \times p))$$

Example 23.3 Use of Transformation Rules



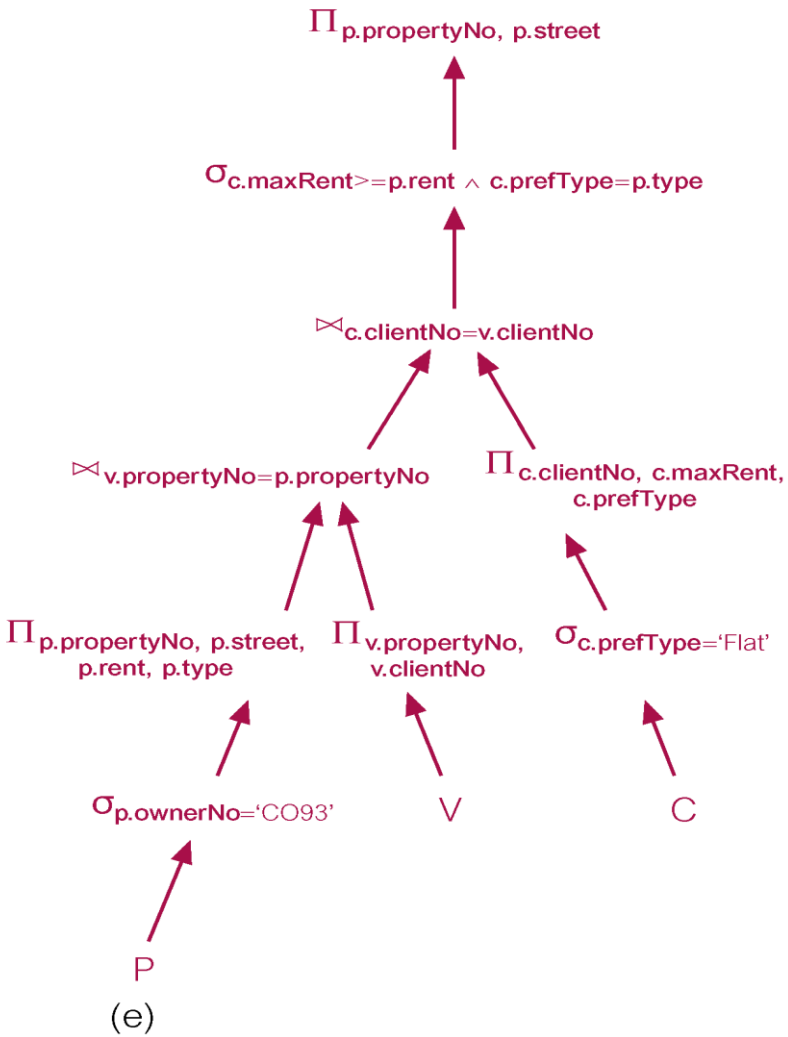
(c)



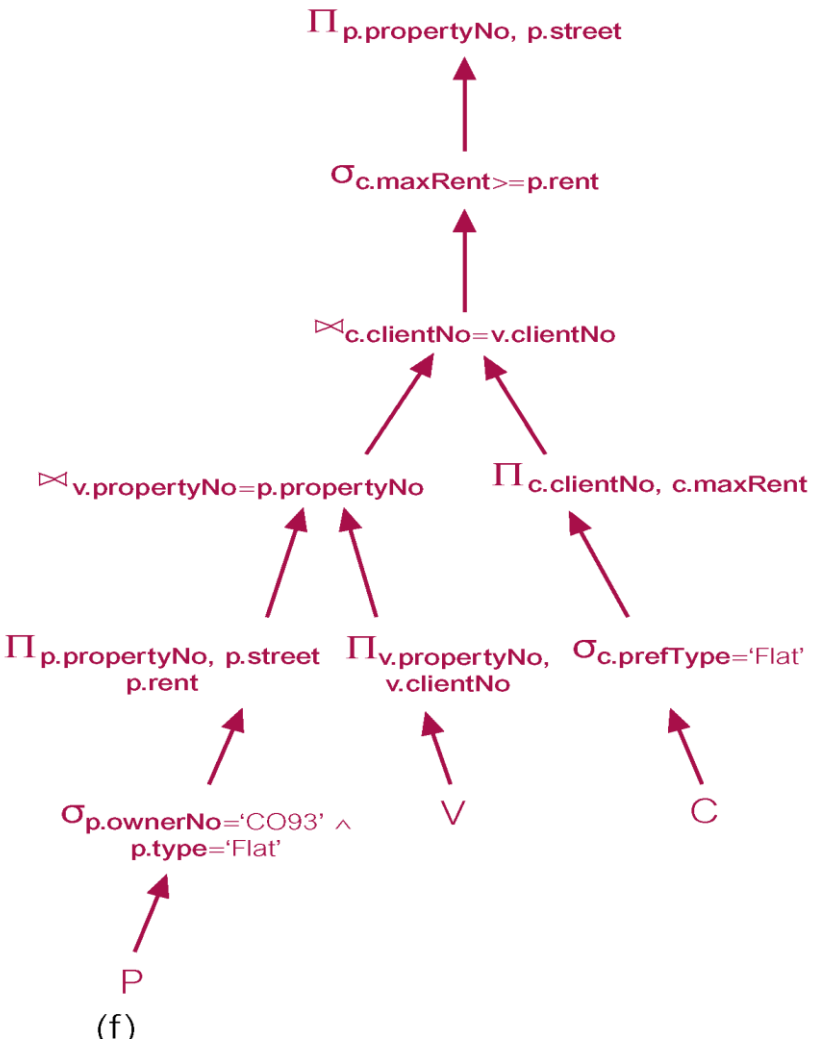
(d)

Pearson Education © 2014

Example 23.3 Use of Transformation Rules



(e)



(f)

Pearson Education © 2014

Heuristical Processing Strategies

- Perform Selection operations as early as possible.
  - Keep predicates on same relation together.
- Combine Cartesian product with subsequent Selection whose predicate represents join condition into a Join operation.
- Use associativity of binary operations to rearrange leaf nodes so leaf nodes with most restrictive Selection operations executed first.

Pearson Education © 2014

Heuristical Processing Strategies

- Perform Projection as early as possible.
  - Keep projection attributes on same relation together.
- Compute common expressions once.
  - If common expression appears more than once, and result not too large, store result and reuse it when required.
  - Useful when querying views, as same expression is used to construct view each time.

Pearson Education © 2014